

Business Case Yulu - Hypothesis Testing

✓ Phase 1 — Data loading and initial inspection

Step 1: Define the business problem

✓ 1. Business Problem Statement

The objective of this analysis is to understand the characteristics of bank customers and explore how customer demographics, financial information, employment details, and previous marketing interactions relate to the outcome recorded in the **y** column.

The analysis will examine the dataset's structure, data quality, variable distributions, and relationships between important attributes. These findings will help identify patterns that may support better understanding of customer responses to bank marketing campaigns.

The exact meaning of the target variable will be confirmed before drawing final conclusions.

Step 2: Check dataset shape, columns, and data types

```
pip install -q gdown
```

```
!gdown "14CV1dU-GnMj2AfkKmsGS-1lc5QfkJiqN"
```

Downloading...

From: <https://drive.google.com/uc?id=14CV1dU-GnMj2AfkKmsGS-1lc5QfkJiqN>

To: /content/bike_sharing.csv

100% 648k/648k [00:00<00:00, 111MB/s]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df =pd.read_csv('bike_sharing.csv')
```

```
yulu = df.copy()
```

```
yulu.head()
```

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windsp
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	
	2011-01-								

Next steps:

[Generate code with yulu](#)
[New interactive sheet](#)

```
# 1. Check the number of rows and columns
print("Dataset Shape:", yulu.shape)
```

```
# 2. Check all column names
print("\nColumn Names:")
print(yulu.columns.tolist())
```

```
# 3. Check data types and non-null values
print("\nDataset Information:")
yulu.info()
```

Dataset Shape: (10886, 12)

Column Names:

```
['datetime', 'season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'hum
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10886 entries, 0 to 10885
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	datetime	10886 non-null	object
1	season	10886 non-null	int64
2	holiday	10886 non-null	int64
3	workingday	10886 non-null	int64
4	weather	10886 non-null	int64
5	temp	10886 non-null	float64
6	atemp	10886 non-null	float64
7	humidity	10886 non-null	int64
8	windspeed	10886 non-null	float64
9	casual	10886 non-null	int64
10	registered	10886 non-null	int64
11	count	10886 non-null	int64

```
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
```

✓ Key observations

1. Dataset size: We have 10,886 records and 12 columns to analyze.
2. Datetime: The datetime column is currently an object. We should convert it to Pandas datetime format so we can analyze dates, hours, and seasonal patterns.
3. Categorical variables: season, holiday, workingday, and weather are stored as integers, but they represent categories or binary indicators. We'll consider converting them to the category data type.
4. Numerical variables: temp, atemp, humidity, windspeed, and count are numerical measurements or counts. casual and registered are also numerical counts.
5. Missing values: All 12 columns show 10,886 non-null entries. This means there are no missing values detected by info().

Step 3: Detect missing values and duplicates

```
# Check missing values
print("Missing values per column:")
print(yulu.isnull().sum())

print("\nTotal missing values:", yulu.isnull().sum().sum())

# Check duplicate records
print("\nTotal duplicate rows:", yulu.duplicated().sum())
```

```
Missing values per column:
datetime      0
season        0
holiday       0
workingday    0
weather       0
temp          0
atemp         0
humidity      0
windspeed    0
casual        0
registered    0
count         0
dtype: int64
```

```
Total missing values: 0
```

```
Total duplicate rows: 0
```

Step 4: Identify categorical columns

```
# Display unique values in each column
for col in yulu.columns:
    print(f"\n{col}: {yulu[col].nunique()} unique values")
    print(yulu[col].unique()[:10])

datetime: 10886 unique values
['2011-01-01 00:00:00' '2011-01-01 01:00:00' '2011-01-01 02:00:00'
 '2011-01-01 03:00:00' '2011-01-01 04:00:00' '2011-01-01 05:00:00'
 '2011-01-01 06:00:00' '2011-01-01 07:00:00' '2011-01-01 08:00:00'
 '2011-01-01 09:00:00']

season: 4 unique values
[1 2 3 4]

holiday: 2 unique values
[0 1]

workingday: 2 unique values
[0 1]

weather: 4 unique values
[1 2 3 4]

temp: 49 unique values
[ 9.84  9.02  8.2  13.12 15.58 14.76 17.22 18.86 18.04 16.4 ]

atemp: 60 unique values
[14.395 13.635 12.88  17.425 19.695 16.665 21.21  22.725 21.97  20.455]

humidity: 89 unique values
[81 80 75 86 76 77 72 82 88 87]

windspeed: 28 unique values
[ 0.      6.0032 16.9979 19.0012 19.9995 12.998  15.0013  8.9981 11.0014
 22.0028]

casual: 309 unique values
[ 3  8  5  0  2  1 12 26 29 47]

registered: 731 unique values
[13 32 27 10  1  0  2  7  6 24]

count: 822 unique values
[16 40 32 13  1  2  3  8 14 36]
```

Step 5: Review descriptive statistics

```
# Statistical summary of numerical columns
display(df.describe())
```

```
# Summary of categorical columns
display(df.describe(include="object"))
```

	season	holiday	workingday	weather	temp	
count	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000
mean	2.506614	0.028569	0.680875	1.418427	20.23086	23.08611
std	1.116174	0.166599	0.466159	0.633839	7.79159	8.18580
min	1.000000	0.000000	0.000000	1.000000	0.82000	0.00000
25%	2.000000	0.000000	0.000000	1.000000	13.94000	16.00000
50%	3.000000	0.000000	1.000000	1.000000	20.50000	24.00000
75%	4.000000	0.000000	1.000000	2.000000	26.24000	31.00000
max	4.000000	1.000000	1.000000	4.000000	41.00000	45.00000
datetime						
count	10886					
unique	10886					
top	2012-12-19 23:00:00					
freq	1					

What do these numbers mean?

- Mean (191.57) is higher than median (145): This suggests the rental-count distribution may be right-skewed. A relatively small number of high-demand hours could pull the average upward.
- Large variation in rentals: The standard deviation is about 181.14, indicating substantial variation in hourly rental counts.
- Casual vs. registered users: Mean casual rentals are approximately 36.02 per hour, while mean registered rentals are approximately 155.55 per hour.
- Temperature: Average temperature is 20.23°C, ranging from 0.82°C to 41°C.
- Humidity: Values range from 0% to 100%, with an average of approximately 61.89%.
- Windspeed: Values range from 0 to approximately 57, with an average of 12.80. The units should be confirmed from the dataset documentation.

Step 6: Convert appropriate columns to category

```
# Step 6: Convert appropriate columns to category

# Convert datetime to proper datetime format
yulu['datetime'] = pd.to_datetime(yulu['datetime'])

# Select categorical columns
categorical_cols = ['season', 'holiday', 'workingday', 'weather']

# Convert them to category data type
for col in categorical_cols:
    yulu[col] = yulu[col].astype('category')

# Verify updated data types
print("Updated data types:")
print(yulu.dtypes)
```

```
Updated data types:
datetime      datetime64[ns]
season        category
holiday       category
workingday    category
weather       category
temp          float64
atemp         float64
humidity      int64
windspeed     float64
casual        int64
registered    int64
count         int64
dtype: object
```

```
# Summary of numerical variables
print("Numerical Summary:")
display(yulu.describe())

# Summary of categorical variables
print("Categorical Summary:")
display(yulu.describe(include='category'))
```

Numerical Summary:

	datetime	temp	atemp	humidity	windspeed	
count	10886	10886.00000	10886.000000	10886.000000	10886.000000	1
mean	2011-12-27 05:56:22.399411968	20.23086	23.655084	61.886460	12.799395	
min	2011-01-01 00:00:00	0.82000	0.760000	0.000000	0.000000	
25%	2011-07-02 07:15:00	13.94000	16.665000	47.000000	7.001500	
50%	2012-01-01 20:30:00	20.50000	24.240000	62.000000	12.998000	
75%	2012-07-01 12:45:00	26.24000	31.060000	77.000000	16.997900	
max	2012-12-19 23:00:00	41.00000	45.455000	100.000000	56.996900	
std	NaN	7.79159	8.474601	19.245033	8.164537	

Categorical Summary:

	season	holiday	workingday	weather
count	10886	10886	10886	10886

✓ Phase 2 — EDA roadmap

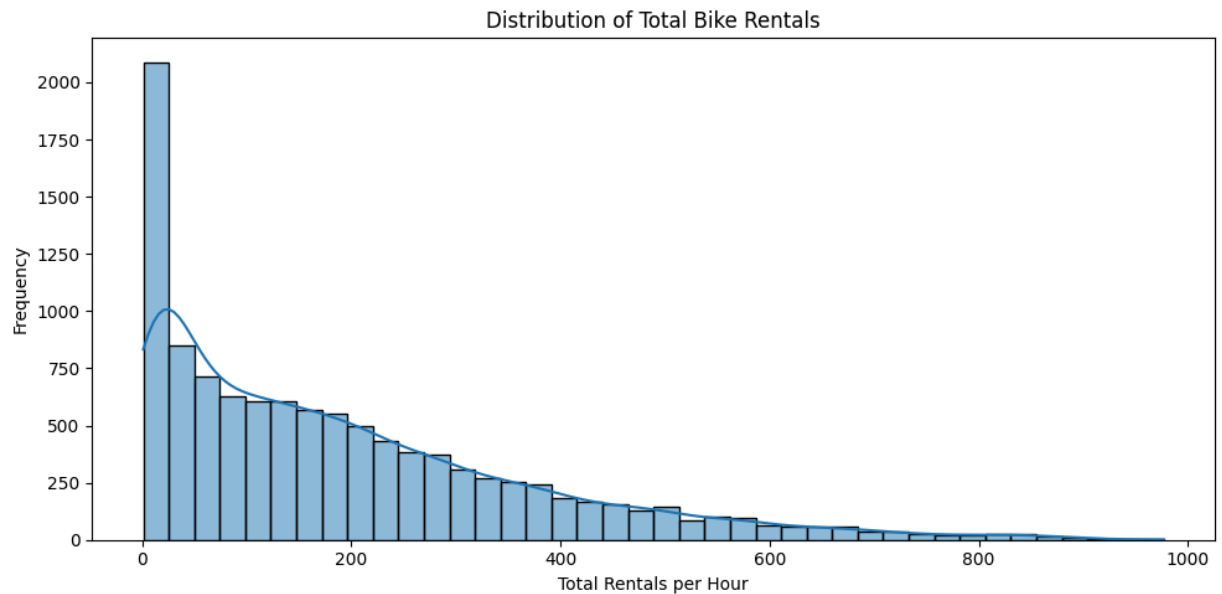
Step 1 — Univariate analysis: Rental count

```
import matplotlib.pyplot as plt
import seaborn as sns

# Distribution of total bike rentals
plt.figure(figsize=(10, 5))

sns.histplot(data=yulu, x='count', bins=40, kde=True)

plt.title('Distribution of Total Bike Rentals')
plt.xlabel('Total Rentals per Hour')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



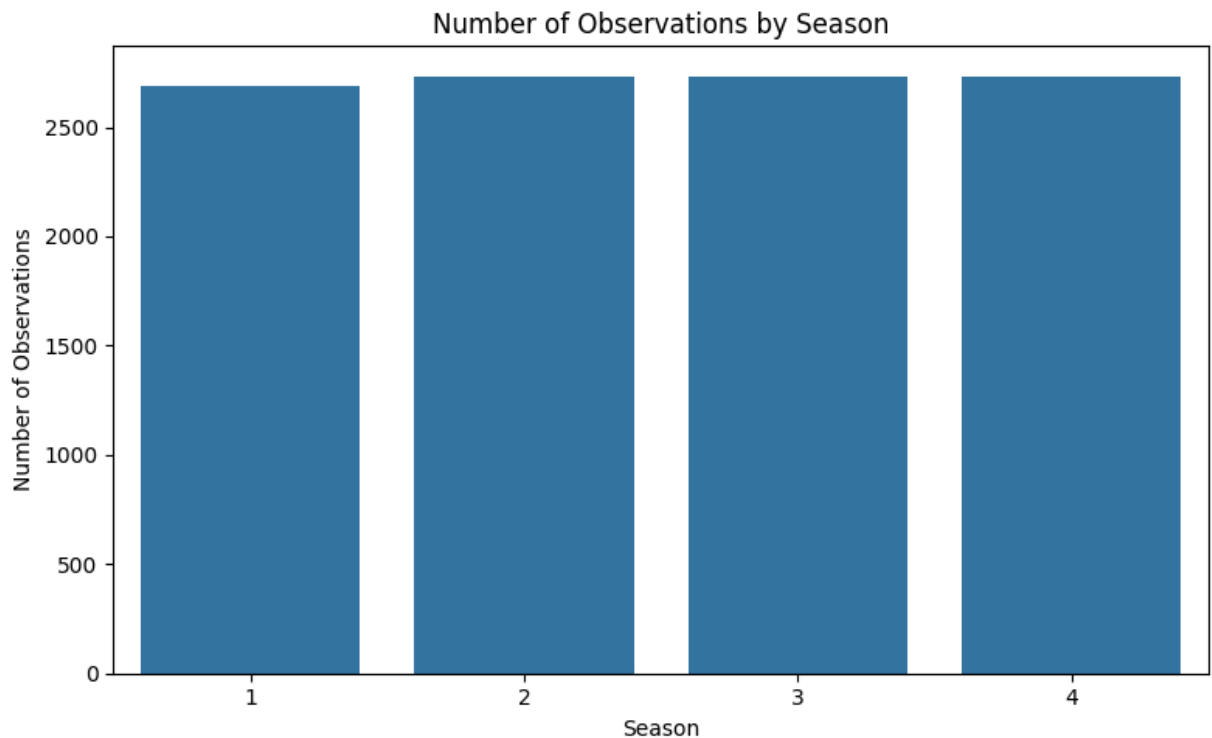
Observation: The distribution of hourly bike rentals is positively (right) skewed. Most observations have relatively low rental counts, while a smaller number of observations show very high demand. This indicates substantial variation in hourly bike rentals. Further analysis is needed to investigate how season, weather, and working days relate to rental demand.

Step 2 — Analyze the categorical variable season

```
plt.figure(figsize=(8, 5))

sns.countplot(data=yulu, x='season')

plt.title('Number of Observations by Season')
plt.xlabel('Season')
plt.ylabel('Number of Observations')
plt.tight_layout()
plt.show()
```



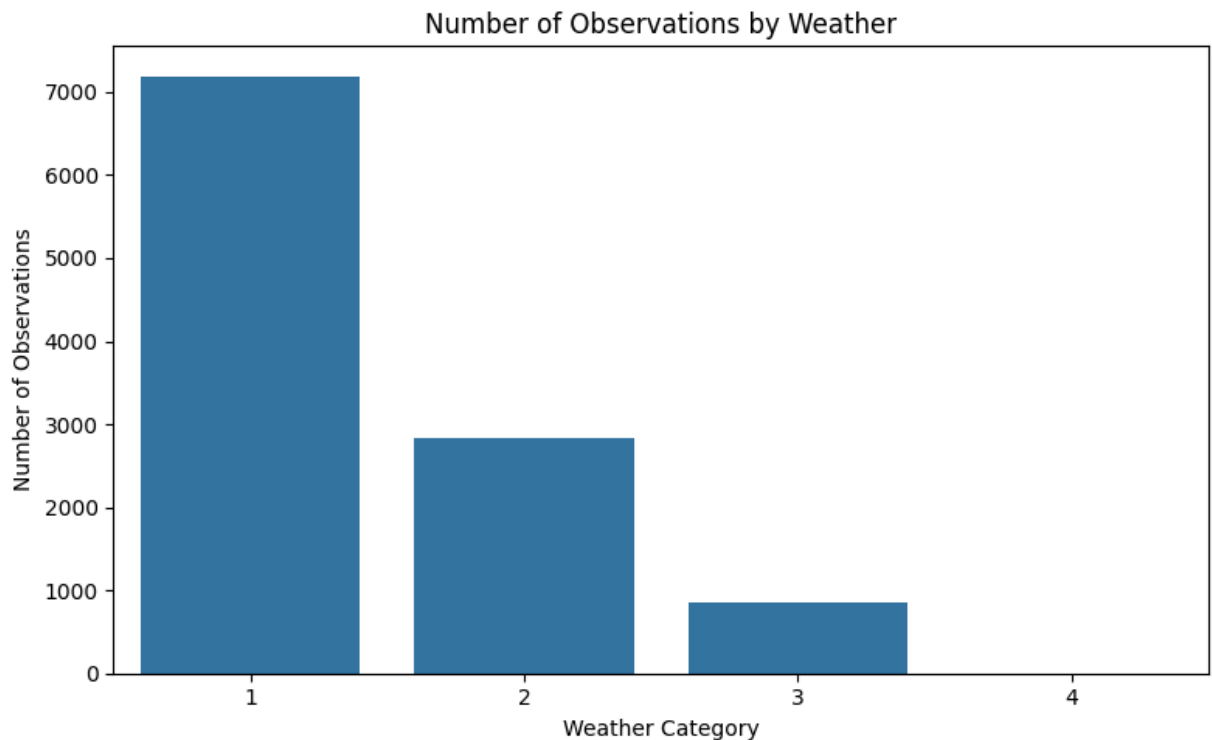
Observation The season countplot shows that observations are distributed almost equally across the four season categories. Therefore, the dataset has a reasonably balanced seasonal representation. However, the distribution of observations alone does not indicate differences in bike rental demand between seasons.

Step 3 — Analyze the weather variable

```
plt.figure(figsize=(8, 5))

sns.countplot(data=yulu, x='weather')

plt.title('Number of Observations by Weather')
plt.xlabel('Weather Category')
plt.ylabel('Number of Observations')
plt.tight_layout()
plt.show()
```



Observation The weather distribution is highly imbalanced. Weather category 1 accounts for most observations, followed by categories 2 and 3. Category 4 is extremely rare, which may affect the reliability of statistical comparisons across weather categories.

4.Final Observation

```
yulu['weather'].value_counts().sort_index()
```

```
yulu['weather'].value_counts().sort_index()
```

	count
weather	
1	7192
2	2834
3	859
4	1

dtype: int64

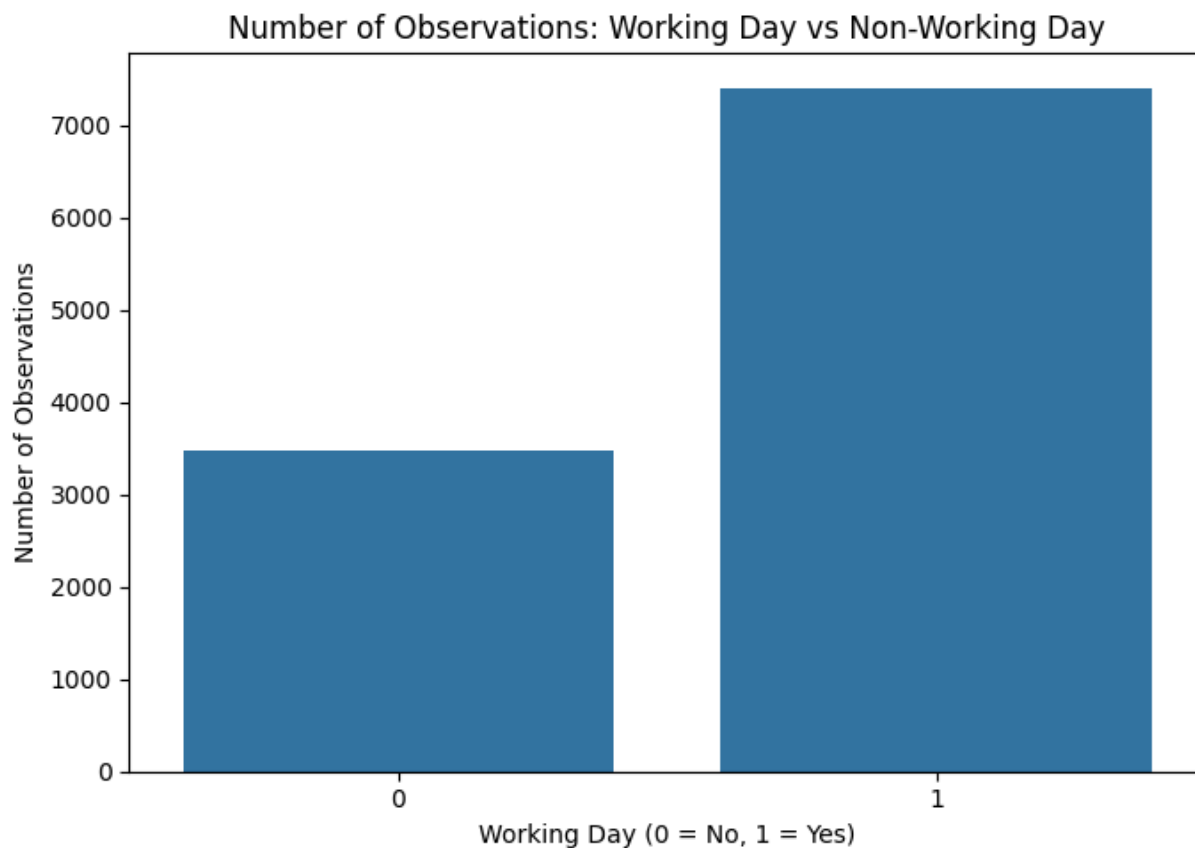
Observation: Weather category 1 accounts for approximately 66.08% of all observations, while category 4 contains only one observation. This creates a highly imbalanced distribution. The rare category may limit the reliability of statistical comparisons, particularly ANOVA, because its within-group variance cannot be estimated from a single observation.

Step 5 — Analyze workingday

```
plt.figure(figsize=(7, 5))

sns.countplot(data=yulu, x='workingday')

plt.title('Number of Observations: Working Day vs Non-Working Day')
plt.xlabel('Working Day (0 = No, 1 = Yes)')
plt.ylabel('Number of Observations')
plt.tight_layout()
plt.show()
```



```
yulu['workingday'].value_counts().sort_index()
```

	count
workingday	
0	3474
1	7412

dtype: int64

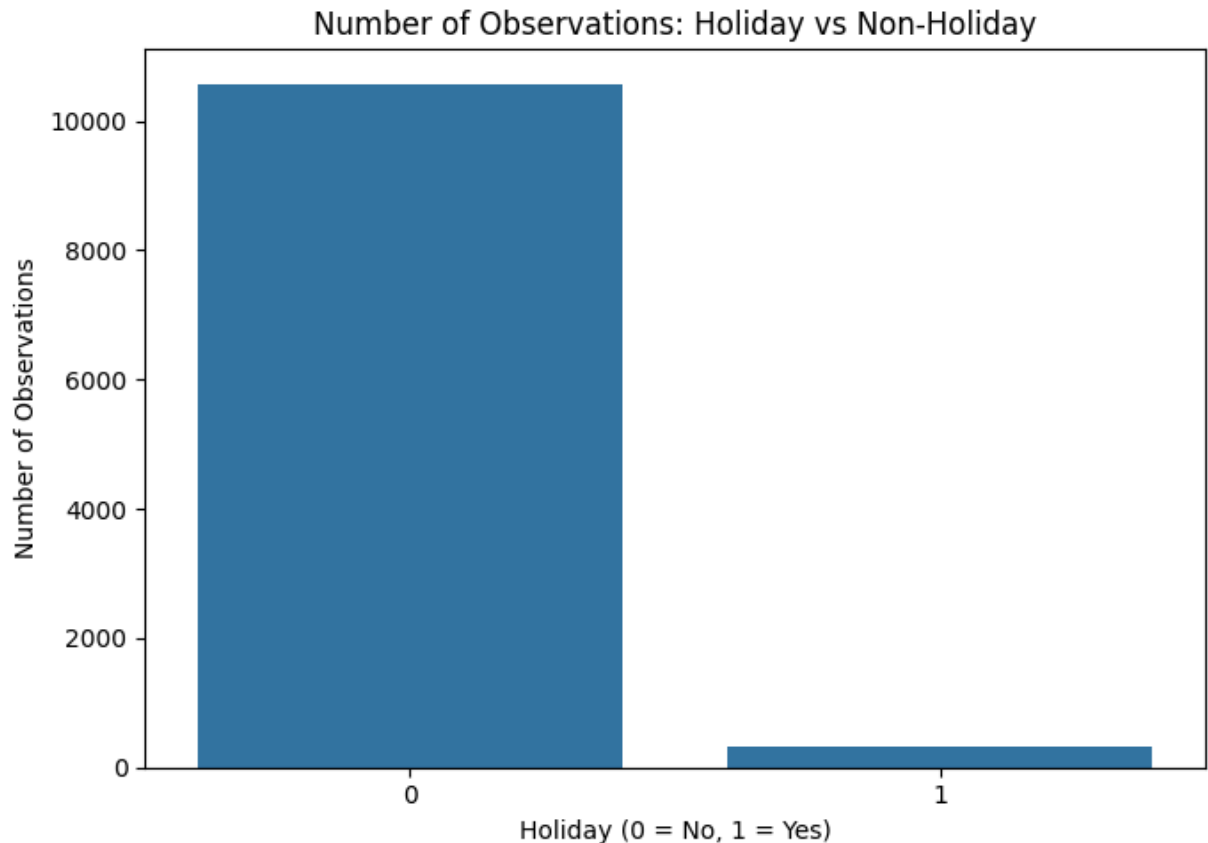
Step 6 — Analyze the holiday variable

```
plt.figure(figsize=(7, 5))

sns.countplot(data=yulu, x='holiday')

plt.title('Number of Observations: Holiday vs Non-Holiday')
plt.xlabel('Holiday (0 = No, 1 = Yes)')
plt.ylabel('Number of Observations')
```

```
plt.tight_layout()  
plt.show()
```



Observation The holiday countplot shows that non-holiday observations substantially outnumber holiday observations. The dataset is highly imbalanced across these categories. Further bivariate analysis is required to determine whether average bike rental demand differs between holidays and non-holidays.

```
yulu['holiday'].value_counts().sort_index()
```

	count
holiday	
0	10575
1	311

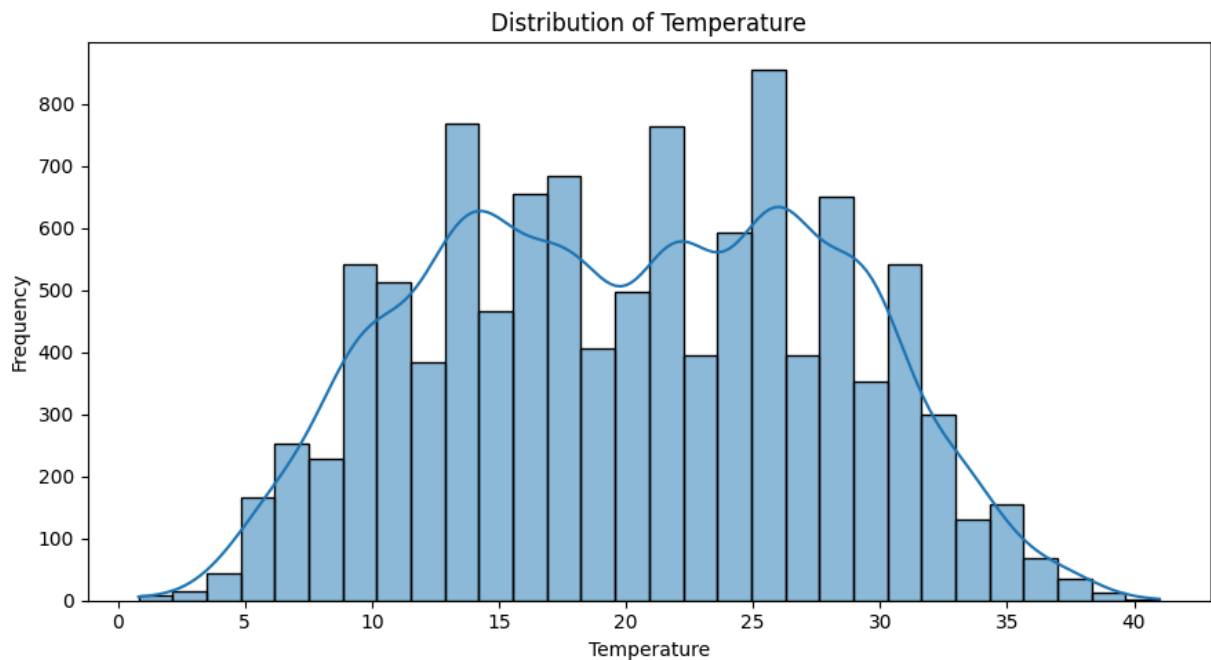
dtype: int64

Step 7 — Analyze the temp variable

```
plt.figure(figsize=(9, 5))

sns.histplot(data=yulu, x='temp', bins=30, kde=True)

plt.title('Distribution of Temperature')
plt.xlabel('Temperature')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



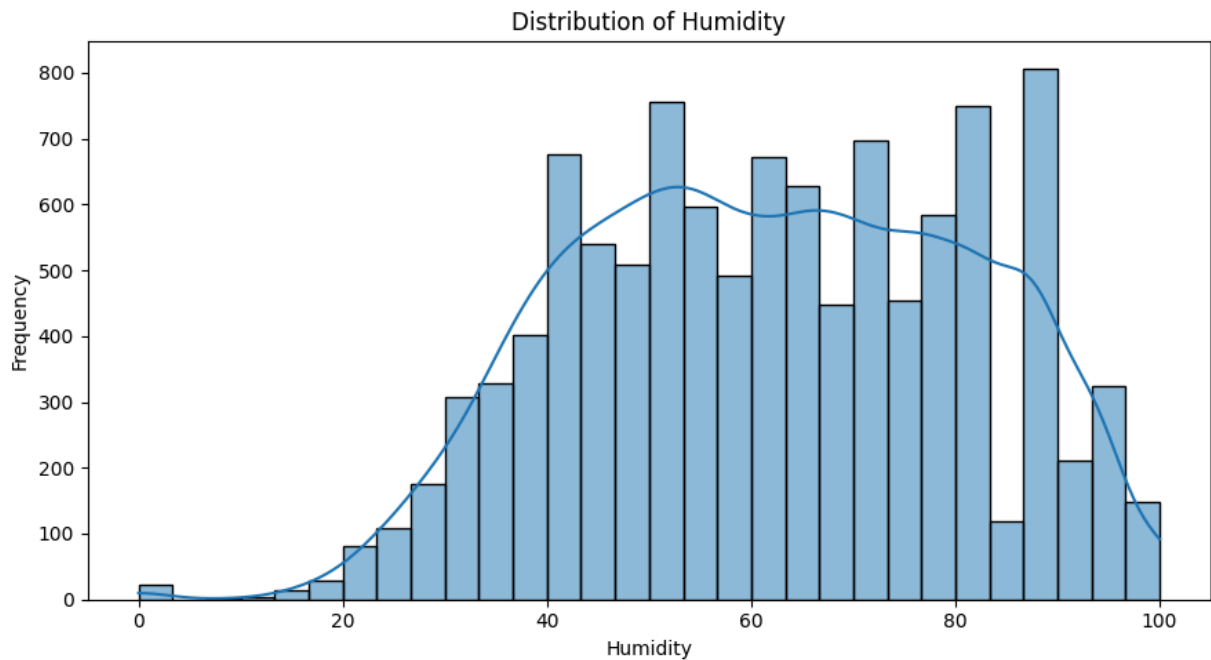
Observation: The temperature distribution spans a broad range, with observations concentrated across several temperature intervals. The histogram does not appear perfectly bell-shaped. Further bivariate analysis is needed to investigate the relationship between temperature and bike rental demand.

Step 8 — Analyze humidity

```
plt.figure(figsize=(9, 5))

sns.histplot(data=yulu, x='humidity', bins=30, kde=True)
```

```
plt.title('Distribution of Humidity')
plt.xlabel('Humidity')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



observation The humidity distribution spans approximately 0–100, with most observations concentrated between roughly 35 and 90. The distribution is irregular and has multiple peaks. A small number of observations have very low humidity values, which should be investigated for potential data-quality issues. Further analysis is required to understand the relationship between humidity and bike rental demand.

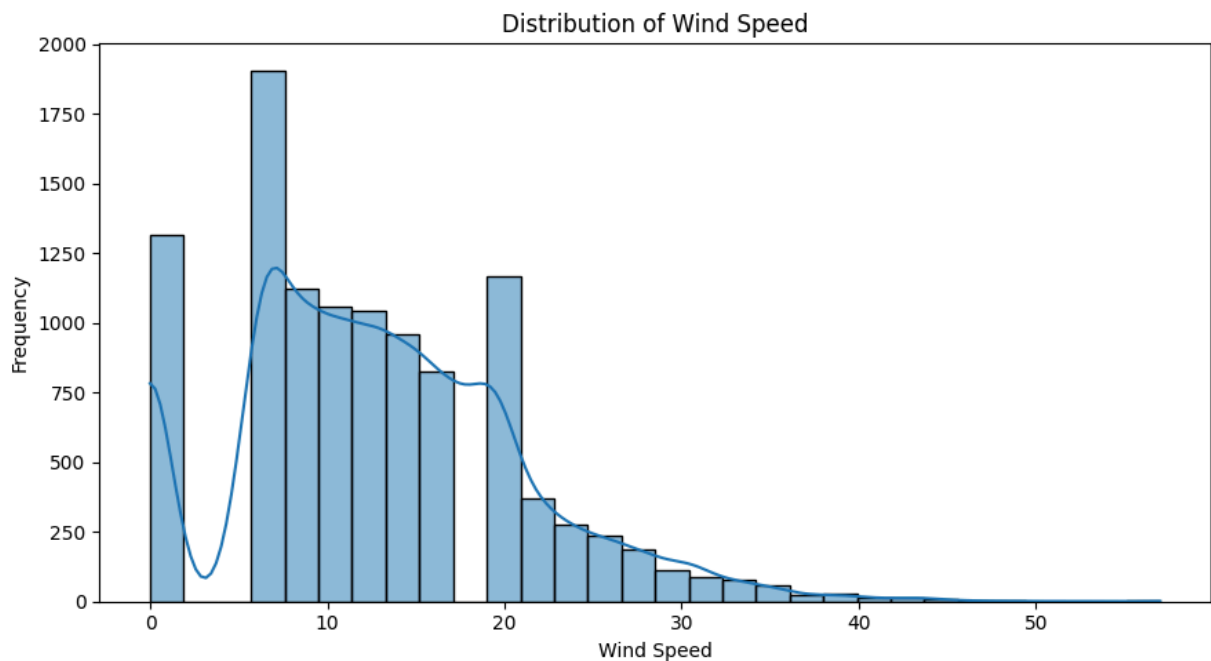
Step 9 — Analyze wind speed

```
plt.figure(figsize=(9, 5))

sns.histplot(data=yulu, x='windspeed', bins=30, kde=True)

plt.title('Distribution of Wind Speed')
plt.xlabel('Wind Speed')
```

```
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



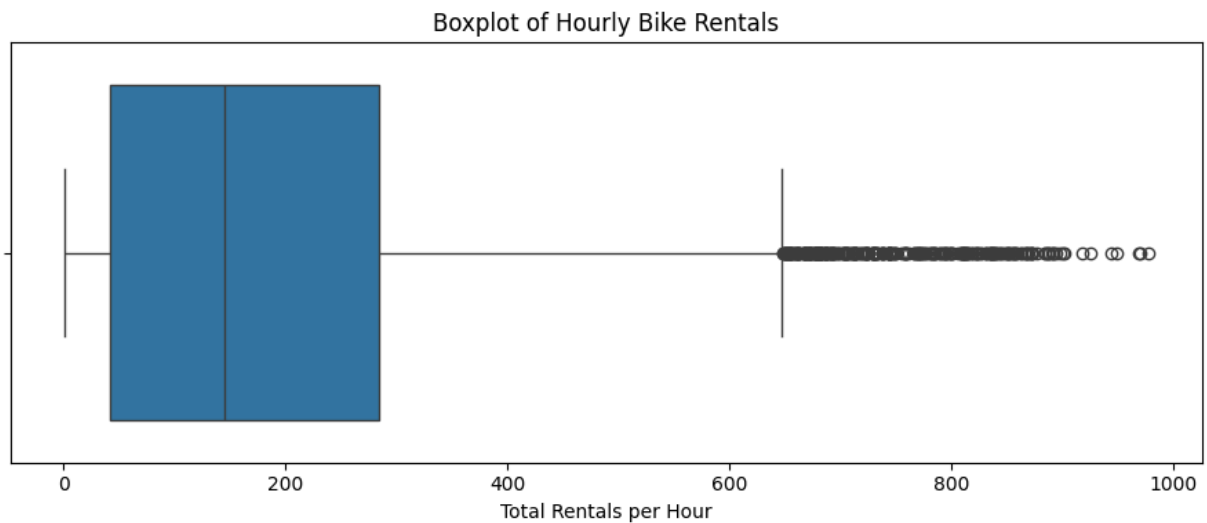
observation Wind speed is positively skewed, with most observations concentrated at relatively low to moderate values and fewer observations at higher values. Several observations have zero wind speed and should be investigated. Further bivariate analysis is required to assess the relationship between wind speed and bike rental demand.

Step 10 — Let's check outliers in our main variable:

```
plt.figure(figsize=(9, 4))

sns.boxplot(data=yulu, x='count')

plt.title('Boxplot of Hourly Bike Rentals')
plt.xlabel('Total Rentals per Hour')
plt.tight_layout()
plt.show()
```



observation The boxplot of hourly bike rentals shows a right-skewed distribution with numerous potential upper outliers. The median rental count is 145, and the middle 50% of observations range from 42 to 284 rentals per hour. The unusually high values may represent genuine peak-demand periods and should be investigated before considering any treatment.

Step 11 — Calculate the IQR and outlier boundaries

```
Q1 = yulu['count'].quantile(0.25)
Q3 = yulu['count'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

outliers = yulu[
    (yulu['count'] < lower_bound) |
    (yulu['count'] > upper_bound)
```

```
]

print("Number of potential outliers:", len(outliers))

Q1: 42.0
Q3: 284.0
IQR: 242.0
Lower Bound: -321.0
Upper Bound: 647.0
Number of potential outliers: 300
```

Step 12 — Inspect the highest rental counts

```
yulu.nlargest(10, 'count')[
    ['datetime', 'season', 'weather',
     'workingday', 'holiday', 'temp', 'count']
]
```

	datetime	season	weather	workingday	holiday	temp	count	
9345	2012-09-12 18:00:00	3	1	1	0	27.06	977	
9320	2012-09-11 17:00:00	3	1	1	0	28.70	970	
9297	2012-09-10 18:00:00	3	1	1	0	25.42	968	
9752	2012-10-10 17:00:00	4	1	1	0	23.78	948	
9896	2012-10-16 17:00:00	4	1	1	0	21.32	943	
9344	2012-09-12 17:00:00	3	1	1	0	28.70	925	
9584	2012-10-03 17:00:00	4	1	1	0	28.70	917	

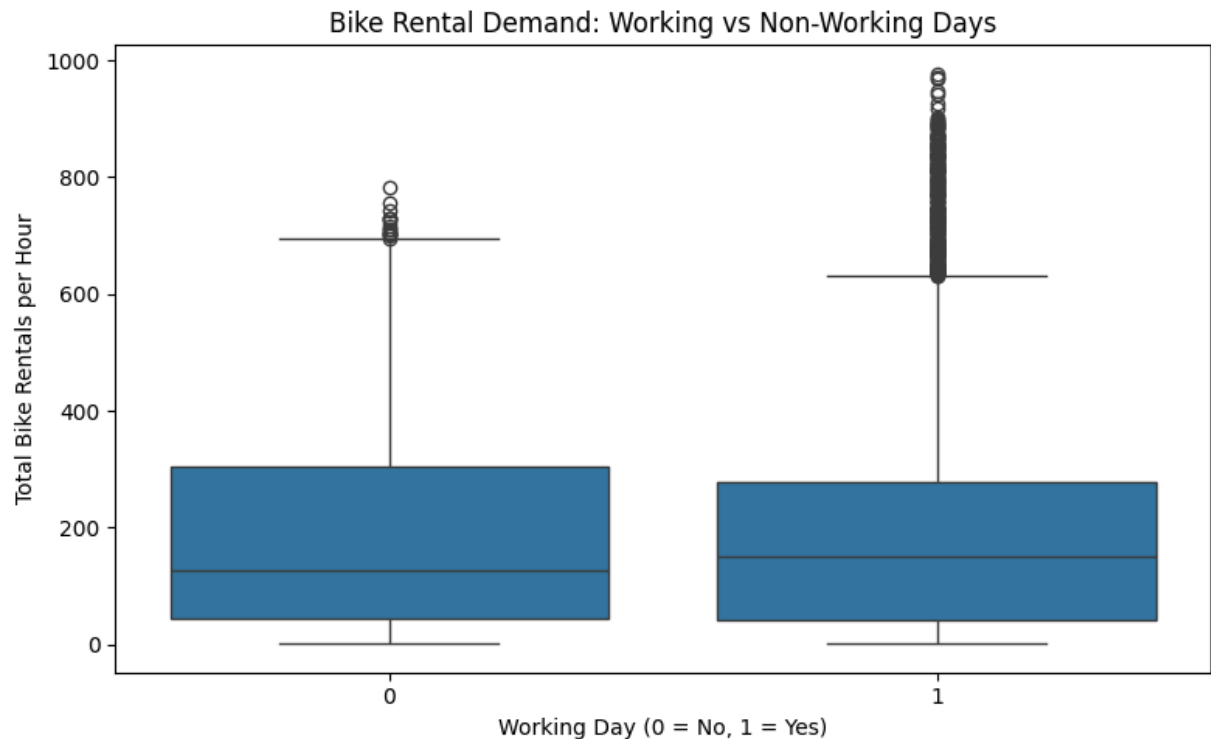
observation Inspection of the 10 highest hourly rental counts shows that all observations occurred at 5 PM or 6 PM on working days and under weather category 1. Rental counts ranged from 897 to 977. This pattern may indicate evening commuting demand, but a comparison across all observations is required to validate this hypothesis.

Step 13 — Now let's connect the data to the business question

```
plt.figure(figsize=(8, 5))

sns.boxplot(data=yulu, x='workingday', y='count')
```

```
plt.title('Bike Rental Demand: Working vs Non-Working Days')
plt.xlabel('Working Day (0 = No, 1 = Yes)')
plt.ylabel('Total Bike Rentals per Hour')
plt.tight_layout()
plt.show()
```



observation The boxplot suggests that the median hourly rental count is somewhat higher on working days than on non-working days. Both groups show considerable variability and potential upper outliers. Since the distributions overlap, a statistical test is needed to determine whether there is sufficient evidence of a difference in mean rental demand.

Step 14 — Compare the actual statistics

```
yulu.groupby('workingday', observed=True)['count'].agg(
    ['count', 'mean', 'median', 'std']
).round(2)
```

	count	mean	median	std	
workingday					
0	3474	188.51	128.0	173.72	
1	7412	193.01	151.0	184.51	

observation The average hourly rental count is 193.01 on working days and 188.51 on non-working days, a difference of 4.50 rentals per hour. The median rental count is also higher on working days. Both groups exhibit substantial variability. A statistical hypothesis test is required to assess whether the difference in mean rental counts is statistically significant.

Step 15 — Formulate our first hypothesis test

```
from scipy.stats import ttest_ind

non_working = yulu.loc[yulu['workingday'] == 0, 'count']
working = yulu.loc[yulu['workingday'] == 1, 'count']

t_stat, p_value = ttest_ind(
    non_working,
    working,
    equal_var=False
)

print("T-statistic:", t_stat)
print("P-value:", p_value)
```

```
T-statistic: -1.2362580418223226
P-value: 0.21640312280695098
```

Hypothesis test: Welch's independent two-sample t-test.

H_0 : The population mean hourly rental counts are equal on working and non-working days.

H_1 : The population mean hourly rental counts differ between working and non-working days.

Results: ($t=-1.326$), ($p=0.2164$), and ($\alpha=0.05$).

Decision: Fail to reject H_0 because the p-value exceeds the significance level.

Conclusion: The test does not provide sufficient evidence of a difference in population mean hourly rental counts between working and non-working days.

Step 16 — Next: ANOVA across seasons

```
yulu.groupby('season', observed=True)['count'].agg(
    ['count', 'mean', 'median', 'std']
).round(2)
```

	count	mean	median	std	
season					
1	2686	116.34	78.0	125.27	
2	2733	215.25	172.0	192.01	
3	2733	234.42	195.0	197.15	
4	2734	198.99	161.0	177.62	

observation The mean hourly rental count varies across seasons. Season 3 has the highest observed mean (234.42), while season 1 has the lowest (116.34). These differences suggest a possible seasonal relationship with rental demand. One-way ANOVA will be used to test whether there is statistically significant evidence of differences in population means.

```
from scipy.stats import f_oneway

groups = [
    yulu.loc[yulu['season'] == s, 'count']
    for s in [1, 2, 3, 4]
]

f_stat, p_value = f_oneway(*groups)
```

```
print("F-statistic:", f_stat)
print("P-value:", p_value)
```

```
F-statistic: 236.94671081032098
P-value: 6.164843386499654e-149
```

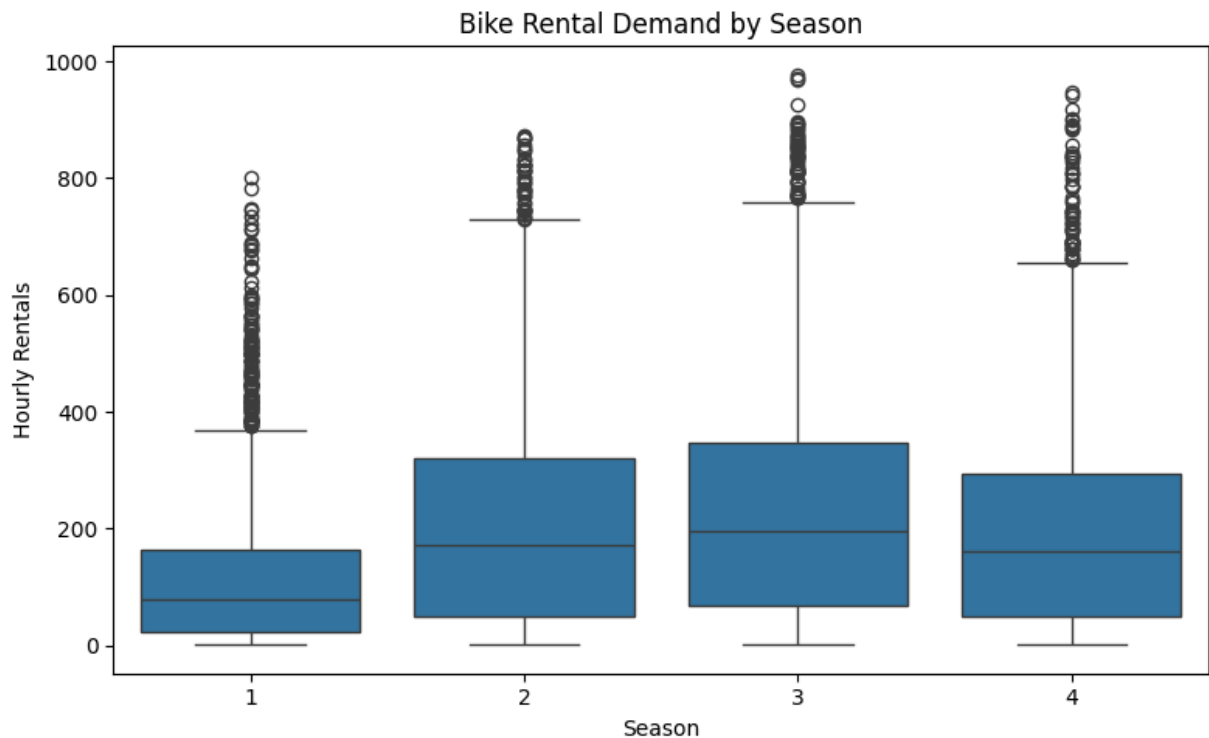
****conclusion: ****One-way ANOVA found a statistically significant difference in mean hourly rental counts across the four seasons ((F $\approx$ 236.95, $p < 0.001$)).

Season vs. rental demand

```
plt.figure(figsize=(8, 5))

sns.boxplot(data=yulu, x='season', y='count')

plt.title('Bike Rental Demand by Season')
plt.xlabel('Season')
plt.ylabel('Hourly Rentals')
plt.tight_layout()
plt.show()
```



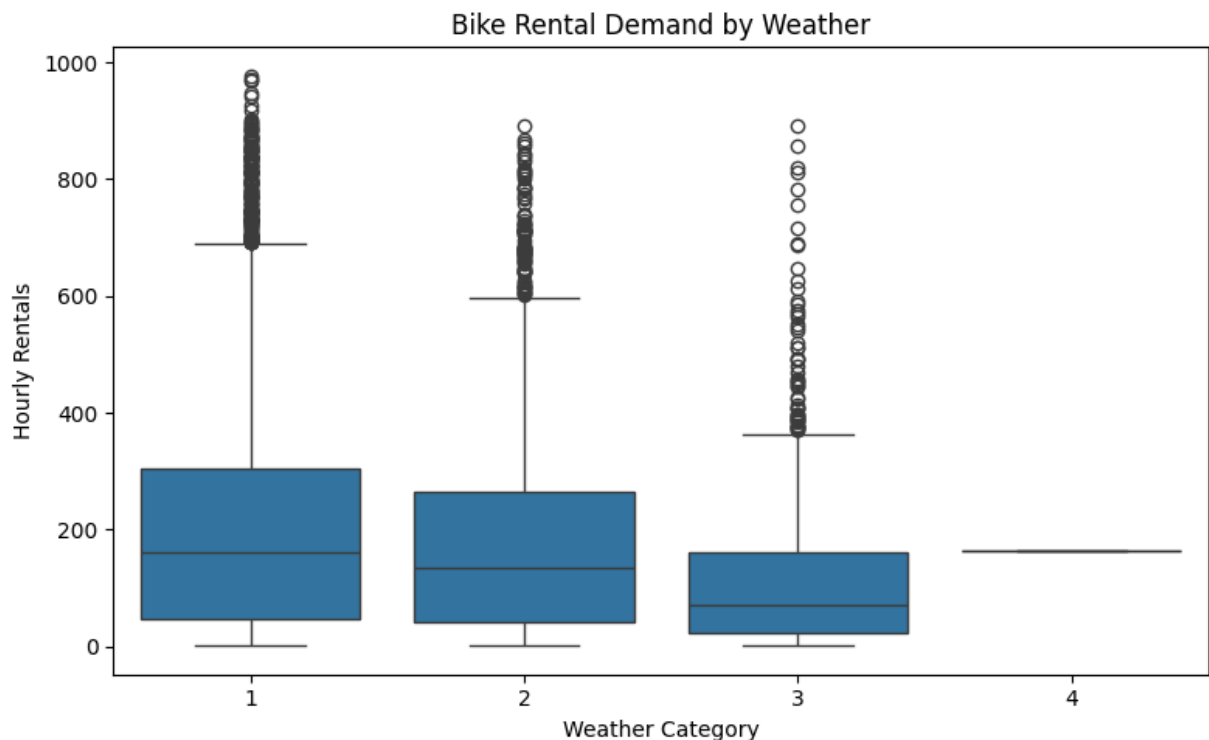
observation: Rental demand varies across seasons, with season 3 showing the highest median and season 1 the lowest. High-demand outliers appear in every season.

Weather vs. rental demand

```
plt.figure(figsize=(8, 5))

sns.boxplot(data=yulu, x='weather', y='count')

plt.title('Bike Rental Demand by Weather')
plt.xlabel('Weather Category')
plt.ylabel('Hourly Rentals')
plt.tight_layout()
plt.show()
```



observation: Median rental demand appears to decrease as the weather category changes from 1 to 3. Category 4 has insufficient observations for a meaningful comparison. This is an association, not proof of causation.

EDA conclusion

- Dataset has 10,886 observations and no missing or duplicate rows.
- Rental counts are right-skewed, with high-demand outliers.
- Demand varies across seasons and weather categories.
- Working-day and non-working-day rental distributions overlap considerably.
- Weather category 4 has only one observation and requires caution.

✓ Phase 3: Hypothesis Testing

1. Independent Two-Sample t-Test

Business question: Is mean hourly bike rental demand different between working days and non-working days? Hypotheses

- (H_0): Mean rental counts are equal between the two groups.
- (H_1): Mean rental counts are different between the two groups. Significance level: ($\alpha = 0.05$)

```
from scipy.stats import ttest_ind

non_working = yulu.loc[yulu['workingday'] == 0, 'count']
working = yulu.loc[yulu['workingday'] == 1, 'count']

t_stat, p_value = ttest_ind(
    non_working, working, equal_var=False
)

print("T-statistic:", t_stat)
print("P-value:", p_value)
```

```
T-statistic: -1.2362580418223226
P-value: 0.21640312280695098
```

Results:

Metric Result: Non-working day mean=188.51

Working day mean=193.01

t-statistic -1.236

p-value 0.2164

Significance level 0.05

Decision and interpretation

Since ($p = 0.2164 > 0.05$), fail to reject (H_0).

Business interpretation: The observed mean rental count is slightly higher on working days, but the test does not provide sufficient evidence of a statistically significant difference in population means.

Assumptions

- Two independent groups.
- Numerical outcome (count).
- Large samples support approximate inference despite non-normality.
- Welch's t-test does not require equal variances.
- Caution: Hourly rental observations may be correlated over time, which can violate the independence assumption.

Status Test 1 complete. Next is Test 2: One-way ANOVA — rental counts across weather categories.

2: One-way ANOVA — rental counts across weather categories.

1. Hypotheses

- H_0 : Mean rental counts are equal across weather categories.
- H_1 : At least one weather category has a different mean rental count.
- Significance level: ($\alpha = 0.05$)

2. Assumptions

3. Independent observations.

4. Numerical outcome: count.

5. Approximately normal residuals within groups, or sufficiently robust inference.

6. Equal population variances for standard one-way ANOVA. Important: Weather category 4 has only one observation, so we'll perform the primary ANOVA on categories 1–3 and document this limitation.

```
from scipy.stats import f_oneway, levene

weather_groups = [
    yulu.loc[yulu['weather'] == w, 'count']
    for w in [1, 2, 3]
]

# Check variance assumption
```

```
lev_stat, lev_p = levene(*weather_groups)

# Perform ANOVA
f_stat, p_value = f_oneway(*weather_groups)

print("Levene p-value:", lev_p)
print("F-statistic:", f_stat)
print("ANOVA p-value:", p_value)
```

```
Levene p-value: 6.198278710731511e-36
F-statistic: 98.28356881946705
ANOVA p-value: 4.976448509904196e-43
```

Test 2 — Results

Significance level 0.05

Decision: Ordinary ANOVA rejects (H_0), since ($p < 0.05$). Important assumption check: Levene's p-value is below 0.05, indicating unequal group variances. So let's verify the result using Welch's ANOVA, which does not assume equal variances.

3. One-way ANOVA: rental counts across seasons.

```
from statsmodels.stats.oneway import anova_oneway

weather_data = yulu[yulu['weather'].isin([1, 2, 3])]

result = anova_oneway(
    data=weather_data['count'],
    groups=weather_data['weather'],
    use_var='unequal'
)

print(result)
```

```
AnovaResult(statistic=np.float64(140.90066217398376), pvalue=np.float64(1.200220e-36))
```

Business interpretation:

Average rental demand differs significantly across weather categories 1–3. Category 1 has the highest observed mean, while category 3 has the lowest. This establishes an association, not causation.

Assumptions:

Levene's test indicated unequal variances, which is why Welch's ANOVA was appropriate. Independence remains an important assumption. Weather category 4 was excluded

because it contains only one observation.

Test 4 — Chi-square test: Weather vs. Season

Question: Are weather category and season independent?

- (H₀): Weather and season are independent.
- (H₁): Weather and season are associated.
- Significance level: ($\alpha=0.05$)

```
from scipy.stats import chi2_contingency

# Exclude weather category 4 (only one observation)
data = yulu[yulu['weather'].isin([1, 2, 3])]

contingency_table = pd.crosstab(
    data['weather'],
    data['season']
)

chi2, p_value, dof, expected = chi2_contingency(
    contingency_table
)

print("Contingency table:")
print(contingency_table)

print("\nChi-square statistic:", chi2)
print("Degrees of freedom:", dof)
print("P-value:", p_value)
print("Minimum expected frequency:", expected.min())
```

```
Contingency table:
season      1      2      3      4
weather
1         1759  1801  1930  1702
2          715   708   604   807
3          211   224   199   225
```

```
Chi-square statistic: 46.101457310732485
Degrees of freedom: 6
P-value: 2.8260014509929403e-08
Minimum expected frequency: 211.88929719797886
```

Hypotheses

- (H₀): Weather and season are independent.
- (H₁): Weather and season are associated.

Assumptions:

Categorical variables, independent observations, and sufficiently large expected cell frequencies. The expected-frequency condition is satisfied. Decision: Since ($p < 0.05$), reject (H_0). Business interpretation: There is statistically significant evidence of an association between weather category and season. This does not establish causation.

Verify the working-day t-test

```
from scipy.stats import ttest_ind

non_working = yulu.loc[yulu['workingday'] == 0, 'count']
working = yulu.loc[yulu['workingday'] == 1, 'count']

print("Non-working days:")
print("Count:", len(non_working))
print("Mean:", non_working.mean())
print("Std:", non_working.std())

print("\nWorking days:")
print("Count:", len(working))
print("Mean:", working.mean())
print("Std:", working.std())

t_stat, p_value = ttest_ind(
    non_working, working, equal_var=False
)

print("\nT-statistic:", t_stat)
print("P-value:", p_value)
```

```
Non-working days:
Count: 3474
Mean: 188.50662061024755
Std: 173.72401532500032
```

```
Working days:
Count: 7412
Mean: 193.01187263896384
Std: 184.5136590421483
```

```
T-statistic: -1.2362580418223226
P-value: 0.21640312280695098
```

Business interpretation:

We don't have sufficient statistical evidence of a difference in mean hourly rentals between working and non-working days. The observed means are similar, with working

days slightly higher.

Phase 4: Business Insights and Recommendations

1. Key Findings

Working days vs. non-working days: Mean hourly rentals were 193.01 on working days and 188.51 on non-working days. Welch's t-test gave a p-value of 0.2164, so the difference was not statistically significant at the 5% level.

Seasonal demand: One-way ANOVA found a statistically significant difference in mean rental counts across seasons ($F = 236.95$, $p < 0.001$). Season 3 had the highest observed mean (234.42), while season 1 had the lowest (116.34).

Weather and demand: Welch's ANOVA found a statistically significant difference in mean rental counts across weather categories 1–3 ($F = 140.90$, $p < 0.001$). Weather category 1 had the highest observed mean (205.24), while category 3 had the lowest (118.85).

Weather and season: The chi-square test found a statistically significant association between weather category and season ($\chi^2 = 46.10$, $df = 6$, $p < 0.001$).

Outliers: The IQR method flagged 300 hourly rental counts above the upper boundary of 647. These observations may represent genuine periods of high demand and should not automatically be removed.

2. Business Recommendations

1. **Plan for seasonal demand:** Use seasonal rental patterns to support bike availability, maintenance scheduling, and operational planning.
2. **Monitor weather conditions:** Track rental demand by weather category to improve short-term demand planning and bike distribution.
3. **Investigate peak-demand periods:** Examine high-rental hours to identify recurring demand patterns and potential opportunities to improve bike availability.
4. **Avoid relying on working-day status alone:** The t-test did not find a statistically significant difference in mean rental counts. Combine working-day information with time, season, and weather when planning operations.

5. **Validate recommendations with further analysis:** Evaluate hourly and daily trends and test whether findings remain consistent across different periods.

✓ 3. Limitations

- The analysis identifies statistical associations and differences but does not establish causation.
- Hourly observations may be correlated over time, which can affect tests that assume independent observations.
- The standard ANOVA variance assumption was violated for weather groups, so Welch's ANOVA was used.
- Weather category 4 contained only one observation and was excluded from the weather ANOVA and the chi-square analysis.
- ANOVA identifies whether at least one group mean differs but does not identify the specific pairs that differ. Post-hoc testing would be needed.
- Statistical significance does not automatically imply a large or commercially important effect.
- Outliers were flagged using the IQR method but were not automatically removed.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.